

TinCan WebMCP

A secure diagnostic channel between AI agents and the websites they operate.

Contest thesis

WebMCP lets agents act on a website. TinCan gives those agents a structured way to report when the website technically succeeds but semantically fails - while the host privately attaches its own browser evidence.

PROJECT	TOOL	TARGET	LICENSE
tincan-webmcp	report_site_issue	WebMCP Challenge	Apache-2.0

Implementation specification - contest edition

Concise build scope, judge-facing demo, architecture, privacy model, acceptance criteria, and submission plan.

1. Executive Brief

TinCan WebMCP is a privacy-safe frontend flight recorder activated by a single WebMCP reporting tool. The agent supplies semantic context - what it expected versus what it observed. The host site privately supplies technical evidence - recent logs, browser errors, network metadata, performance signals, application context, and trace correlation.

The “aha” moment

Traditional monitoring can see “HTTP 200, no JavaScript errors, healthy performance.” The agent can still know the task failed: “I asked for 20 seats; the website now shows 19.” TinCan joins those two truths into one issue.

MVP promise

- One WebMCP tool: ``report_site_issue``.
- A 60-second in-memory flight recorder for host-page diagnostics.
- No raw diagnostics are returned to the agent.
- A familiar Sentry-style Issue Details view for developers/judges.
- Optional OpenTelemetry trace/request correlation.
- A polished SaaS demo where all technical signals look healthy but the business outcome is wrong.

Not the product

TinCan is not another full RUM platform, not DevTools-over-WebMCP, and not a tool that lets agents read browser logs. It is a narrow agent-to-site diagnostic channel with host-controlled evidence collection.

2. Contest Strategy

Build for the judging rubric, not for maximum SDK surface area.

Judging criterion	How TinCan scores	Proof in demo
WebMCP Leverage	WebMCP is the core semantic-reporting channel, not a cosmetic integration.	Agent discovers and calls <code>`report_site_issue`</code> after verifying a wrong outcome.
Execution	Complete SaaS workflow plus Issues UI - not a console demo.	Judge performs action, sees mismatch, agent reports it, developer opens the correlated issue.
Potential Impact	Addresses semantic failures that RUM/APM cannot infer.	All requests are 200 and JS errors are zero, yet the issue is real.
Creativity & Ambition	Turns agents into a new observability signal while preserving site security.	Agent intent + browser evidence + backend trace = semantic incident.

Official submission constraints

Live WebMCP-enabled app; public open-source repository; clear written explanation; public YouTube demo under 3 minutes with audio; judges must be able to access the live app. Deadline: Sep 3, 2026 at 1:00 PM PDT (3:00 PM Chicago/CDT).

3. Judge-Facing Product Experience

Demo application

Build a polished fictional SaaS subscription portal. It should feel like a real product, but stay intentionally small enough that the TinCan concept remains obvious.

Starting state	Plan: Business; Seats: 10
Agent task	“Upgrade this account to 20 seats.”
Injected defect	Mutation claims success, but persisted/read-back state becomes 19 seats.
Technical state	All API calls return 200; no JS exception; browser performance is healthy.
Agent verification	Agent calls `get_subscription()` and observes 19.
TinCan action	Agent calls `report_site_issue(expected="20 seats", observed="19 seats")`.
Judge payoff	Issues page shows semantic failure plus host-only browser evidence and trace context.

Why expected vs. observed works

WebMCP does not infer the mismatch. The agent carries user intent and compares it with structured site observations. TinCan simply provides the reporting channel.

```
User: "Upgrade to 20 seats"
|
Agent intent: expected = 20
|
change_seat_count({ seats: 20 })
|
get_subscription() -> { seatCount: 19 }
|
Agent detects mismatch
|
report_site_issue({ expected: "20 seats", observed: "19 seats" })
```

4. Issue UX

Use a familiar error-investigation pattern instead of inventing a monitoring dashboard.

The developer-facing experience should resemble a Sentry issue detail page, with a breadcrumb timeline and DevTools-like evidence tabs. This reduces judge learning time and makes TinCan feel production-ready.

Issue Details layout

```
TC-1042                                BLOCKING
Subscription upgrade produced incorrect seat count
Agent reported · Production · acme-saas 1.4.2 · /settings/billing
```

```

AGENT OBSERVATION
Expected: 20 seats      Observed: 19 seats
Operation: change_seat_count  Confidence: 99%

SEMANTIC FAILURE
No technical failure detected.
✓ 4 HTTP requests succeeded  ✓ 0 JS exceptions  ✓ Performance healthy

BREADCRUMBS                                     last 12 sec
12:41:06  Navigation  /settings/billing
12:41:09  POST       /api/subscription      200    348 ms
12:41:10  Console    WARN Seat reconciliation mismatch
12:41:11  GET       /api/subscription      200    92 ms
12:41:18  Agent      Site issue reported

[ Console ] [ Network ] [ Performance ] [ Trace ]

```

UI rule

Do not lead with charts. Lead with one issue, one semantic mismatch, and the exact evidence around it. The memorable visual is: “No technical failure detected” directly above “Expected 20 / Observed 19.”

5. WebMCP Contract

Public tool

```

document.modelContext.registerTool({
  name: "report_site_issue",
  description: "Report a problem observed in the current website...",
  inputSchema: REPORT_SITE_ISSUE_SCHEMA,
  execute: async (input) => {
    const result = await tincan.reportIssue(input);
    return JSON.stringify({
      status: result.status,
      incidentId: result.incidentId
    });
  }
});

```

Keep this concrete registration easy to find in the public repository. The reusable SDK may hide WebMCP behind an adapter internally, but the challenge implementation should visibly demonstrate genuine `document.modelContext.registerTool()` usage.

Input

```

type ReportSiteIssueInput = {
  category:
    | "wrong_result"
    | "unexpected_behavior"
    | "action_failed"
    | "network_failure"
    | "performance"
    | "ui_state_mismatch"
    | "other";

  severity: "info" | "degraded" | "blocking";
  summary: string; // <= 300 chars
  description?: string; // <= 2000 chars
  expected?: string; // <= 1000 chars
  observed?: string; // <= 1000 chars
  operation?: string; // <= 200 chars

```

```
confidence?: number;    // 0..1
};
```

Agent-visible output

```
{
  "status": "reported",
  "incidentId": "INC-1042"
}
```

Never return console logs, stack traces, request metadata, application state, or trace IDs to the agent.

6. Frontend Flight Recorder

The flight recorder is a bounded, memory-only ring buffer for the host website. It continuously captures a small amount of safe diagnostic metadata and freezes a snapshot when the agent reports an issue.

Signal	Capture	Default privacy rule
JS errors	`error`, `unhandledrejection`	Sanitize message/stack; bounded strings.
Console	`warn`, `error`	Bounded serializer; no arbitrary object dumping.
Network	fetch/XHR method, path, status, duration	No request/response bodies; no auth/cookie headers.
Resources	failed script/style/image/font metadata	No DOM contents.
Performance	navigation summaries, long tasks	No full performance timeline.
Web Vitals	CLS, INP, LCP, FCP, TTFB	Allowlist safe attribution only.
App context	Host-defined context providers	Executed only at issue time; size/time bounded.
Correlation	Trace/span/request IDs when safe	Host-controlled allowlist only.
Window	60 seconds	
Max events	500	
Max memory	1 MB	
Max incident payload	256 KB	
Persistence	None in MVP	
Eviction	FIFO by age/count/bytes	

7. Privacy & Security Model

Non-negotiable trust boundary

The agent reports meaning. The site controls evidence. Diagnostic data goes to the site backend, not back to the agent.

Never capture automatically

- Cookies or Authorization headers
- Request or response bodies
- Form field values, keystrokes, clipboard
- localStorage, sessionStorage, IndexedDB contents
- DOM text/HTML or screenshots
- Access tokens, refresh tokens, JWTs, API keys, private keys

Sanitization

- Strip URL origin, query string, fragment, and credentials by default.
- Mask UUID/long numeric path identifiers by default; allow host override.
- Recursively redact sensitive keys such as password, token, authorization, cookie, api_key, jwt, private_key.
- Conservatively detect obvious bearer tokens, JWT-like strings, payment-card-like values, and PEM private-key markers.
- Repeat sanitization server-side; never trust the browser as the only privacy layer.
- Treat all agent text as inert, untrusted data. It cannot alter capture scope, destination, headers, or privacy policy.

8. Implementation Architecture

```
Host Web App
|
+-- TinCan Browser SDK
|   +-- collectors
|   +-- sanitizer
|   +-- Frontend Flight Recorder
|   +-- app context providers
|   +-- correlation adapter
|   +-- incident assembler
|   `-- HTTP transport
|
+-- WebMCP Adapter
|   `-- report_site_issue
|
|-- Demo business tools
|   +-- change_seat_count
|   `-- get_subscription
|
POST /_tincan/issues
|
+-- validate + re-sanitize + rate-limit
+-- fingerprint + semanticOnly classification
+-- persist/stream issue
`-- optional OTel span/event/link
```

Packages

```
@tincan-webmcp/browser
@tincan-webmcp/webmcp
```

```
@tincan-webmcp/server
@tincan-webmcp/otel
```

```
apps/demo-saas
apps/issues-ui
```

Core browser functionality must remain framework-independent. Use TypeScript strict mode. Recommended demo stack: Vite + React for speed, Node/Express for the reference backend, Vitest + Playwright for tests.

9. Incident Model & Semantic Classification

```
type TinCanIncident = {
  schemaVersion: "1.0";
  incident: {
    id: string;
    timestamp: string;
    category: string;
    severity: string;
    summary: string;
    expected?: string;
    observed?: string;
    operation?: string;
    confidence?: number;
  };
  application: { name: string; version?: string; environment?: string };
  page: { pathname: string; visibilityState: string };
  diagnostics: {
    events: DiagnosticEvent[];
    context?: Record<string, unknown>;
    truncated?: boolean;
  };
  correlation?: { traceId?: string; spanId?: string; requestIds?: string[] };
  classification: { semanticOnly: boolean };
};
```

A semantic-only incident is the signature TinCan case:

```
semanticOnly =
  issue.severity !== "info" &&
  jsErrorCount === 0 &&
  failedNetworkCount === 0;
```

10. Backend & OpenTelemetry

Reference endpoint

```
POST /_tincan/issues
Content-Type: application/json
credentials: same-origin
```

- Validate schema and content type.
- Enforce 256 KB body limit and server-side rate limiting.
- Re-sanitize sensitive data.
- Generate server incident ID and stable fingerprint.
- Classify semanticOnly.
- Persist via callback; no database is required by the SDK.
- If OTel is configured, create `tincan.site.issue` and link it to the originating trace when available.

OTel is optional. TinCan should consume existing trace context rather than initialize a competing provider. Useful safe attributes: `tincan.issue.category`, `tincan.issue.severity`, `service.name`, `service.version`, `deployment.environment.name`, and `url.path`. Free-form agent prose belongs in logs/span events, not metric labels.

11. Build Plan

Prioritize the end-to-end contest story before SDK breadth.

Priority	Deliverable	Must prove	Defer if needed
P0	Polished demo SaaS + real WebMCP business tools	Agent can act and verify state	Additional business scenarios
P0	`report_site_issue` + flight recorder	Semantic mismatch becomes a correlated issue	Advanced collectors
P0	Sentry-style Issue Details UI	Judge immediately understands evidence	Charts/dashboarding
P1	Privacy sanitizer + server validation	No sensitive payload leakage	Advanced PII detection
P1	OTel/request correlation	Issue links to distributed execution	Vendor-specific exporters
P1	Tests + public README	Runnable, coherent OSS project	Framework adapters
P2	Extra collector polish	More evidence depth	Session replay, screenshots, offline queue

12. Release-Blocking Acceptance Tests

- `report\_site\_issue` is discoverable in a WebMCP-capable browser.
- The canonical 20→19 demo creates an issue even with HTTP 200 everywhere and zero JS exceptions.
- Agent receives only status + incident ID; no raw diagnostics.
- Authorization, Cookie, JWT, API key, password, payment-card-like values, and URL query secrets are redacted.
- Request/response bodies never appear in diagnostic payloads.
- TinCan instrumentation does not change fetch return values/errors, XHR behavior, console behavior, or navigation.
- Ring buffer enforces time, count, and byte limits.
- Host page continues to function if a collector fails or WebMCP is unavailable.
- Server independently validates and re-sanitizes payloads.
- Live demo has a one-click Reset Demo action.

Testing stack

Vitest for unit tests; Playwright for browser/e2e; strict TypeScript; privacy tests block release.

13. Demo Video Plan

Target 2:30-2:45, leaving margin under the 3-minute limit.

Time	What judge sees	Narration point
0:00-0:15	One-line problem + TinCan logo	“Agents can notice failures traditional monitoring cannot.”
0:15-0:40	Demo SaaS: 10 seats; agent asked for 20	Show WebMCP-powered action, not manual UI clicking.
0:40-1:05	Mutation says success; read-back shows 19	Explain that HTTP is 200 and no JS error occurred.
1:05-1:25	Agent calls `report_site_issue`	Expected=20, observed=19, blocking.
1:25-2:05	Issue Details page	Show semantic-failure banner, breadcrumbs, Console/Network/Performance/Trace .
2:05-2:25	Tiny architecture visual	Agent supplies meaning; site supplies private evidence.
2:25-2:40	Close on impact	“Semantic observability for the agentic web.”

14. Submission Checklist

- Working live URL accessible in ChatGPT in-app browser or Chrome with WebMCP enabled.
- Public GitHub/GitLab/Bitbucket repository with all source, assets, and run instructions.
- Open-source LICENSE file visible/detectable on repository page.
- Actual `document.modelContext.registerTool(...)` implementation easy to find in repository.
- README clearly answers: why WebMCP, better UX, what people+agents can now do, and how WebMCP is implemented.
- Public YouTube demo under 3 minutes with audio.
- Demo remains free and unrestricted through judging period.
- English submission materials.
- Commit history clearly shows hackathon-period implementation work.
- Submit before Sep 3, 2026 1:00 PM PDT / 3:00 PM CDT.

Contest focus

Do not spend the final hours building more telemetry. Spend them making the single semantic-failure story flawless, fast, visually familiar, and impossible to misunderstand.

15. Definition of Done

TinCan is contest-ready when a judge can experience this complete loop without reading documentation:

```
User intent
↓
WebMCP action
↓
Structured verification
↓
Agent detects semantic mismatch
↓
report_site_issue
↓
Host flight recorder snapshots private evidence
↓
Issue Details: Expected 20 / Observed 19
↓
No technical failure detected
↓
Correlated browser + trace evidence
```

One-sentence MVP: Build a privacy-safe frontend flight recorder and WebMCP reporting channel that lets an AI agent report a semantic host-site failure, causing the website to privately attach recent sanitized browser diagnostics and trace context to a familiar developer issue - without exposing those diagnostics to the agent.

16. Verified References

WebMCP Challenge Official Rules - Submission period, live URL, public OSS repository, <3-minute YouTube demo, judging criteria, access requirements. [Source](#)

WebMCP Draft Community Group Report - 19 Aug 2026 - Current `document.modelContext.registerTool()` API, tool schemas, execute callbacks, security model. [Source](#)

Sentry Developer Quick Reference / Issue concepts - Familiar issue-details and breadcrumb mental model used for the TinCan investigation UI. [Source](#)

Chrome DevTools documentation - Familiar Console / Network / Performance evidence organization. [Source](#)

OpenTelemetry JavaScript / Logs specification - Optional browser/backend trace correlation and telemetry interoperability. [Source](#)

Google web-vitals - Core Web Vitals collection without reimplementing browser metric algorithms. [Source](#)

Source verification date: Aug 26, 2026.